

模块七: 智能体开发实战

第二十六讲: 多模型 Pipeline——强化智能体的“推理”大脑

[解释]

本讲概览

本讲在 L25 的基础上做增量升级。L25 中我们用 VLM 看图算出了答案“12”，但只返回一个干巴巴的数字。本讲将引入两个新模型：DeepSeek 负责把答案包装成口语化的播报文案，CosyVoice 负责将文案转成语音。三个模型各管一段，串成流水线。

本讲的重点不在于某个模型的法，而在于“流水线”这种架构思想：把一个复杂任务拆成多个独立环节，每个环节由最合适的工具完成。

本节目标: 智能体的迭代进化

在 L25 中，我们已经完成了第一步的“视觉解题”。这节课，我们将通过代码的**增量迭代**，在不破坏原有解题逻辑的前提下，为程序接入语言与声音模块：

- **[理解]** 阐述单体大模型在复杂任务中的职能错配问题与解耦协作优势。
- **[应用]** 构建 `VLM -> LLM -> TTS` 三阶段多模型协作流水线。
- **[分析]** 当输入内容发生变化时，验证流水线能否在不修改代码的前提下正常工作。

能力定位对齐

本讲是对智能体“感-推-行”架构的扩展。不再由一个模型包办所有工作，而是让多个专业模型组成协作流水线。

[解释]

教学目标解读

本页列出了三个层次的目标。“理解”层要求知道为什么要拆分模型；“应用”层要求动手搭出三阶段流水线；“分析”层要求验证系统的泛化能力——即换了输入内容后，不改代码，系统还能正常运行。页面右侧的“能力定位对齐”提示框说明了本讲在整个模块七中的位置：它是对 L25“感知-推理-行动”架构的扩展，把单一的推理环节拆分为多个协作模型。

一、痛点分析

单体模型的局限与架构升级

[解释]

章节定位：为什么要拆分模型

本章通过 What-If 设问（如果要换成方言语音怎么办？）引出单体模型的耦合问题，为后续引入多模型流水线架构提供动机。

单体模型 vs 多模型流水线

🤔 **启发思考 (What-If)**: 假设在系统上线后, 需求方突然要求将语音播报换成特定的“四川方言”。如果我们的系统使用的是一个不可拆解的“全能单体模型”, 你将如何实现这个需求? 重训练的成本有多大?

单体 VLM 模型 (如 GPT-5, Gemini 3等)

- **优点**: 开发与部署简单; 上下文统一, 不易丢失状态; 模块间无网络传输延迟。
- **缺点**: 耦合度极高, 难以单独定制某一环节 (如难以替换特定的方言语音); 长文本推理和语音合成的专业深度往往不如垂类模型。

多模型流水线 (Pipeline)

- **优点**: 灵活组装, 可按需选择各环节最合适的垂类模型 (视觉 + 推理 + 语音); 可针对不同模块独立优化成本。
- **缺点**: 架构复杂度增加; 多次 API 调用会引入额外的网络延迟。

工程抉择: 没有银弹

业界并没有绝对完美的架构。在本次“智能学伴”实战中, 由于我们需要接入具有丰富情感表现的语音能力, 且希望控制逻辑推理成本, 因此我们选择**多模型流水线**, 让各环节由最合适的专业模型分别承担。

[解释]

单体模型与多模型流水线的权衡

本页是一道典型的工程权衡分析题。单体模型 (如 GPT-5) 的优势是简单: 一个 API 调用就能完成语言、视觉或语音全部工作, 不用操心模块间的数据传递。缺点是耦合度高——如果客户要求把语音换成四川方言, 你无法单独替换语音模块, 只能等模型厂商更新。

多模型流水线的优势是灵活: 视觉用 Qwen-VL、推理用 DeepSeek、语音用 CosyVoice, 各取所长。缺点是架构复杂度增加, 多次网络请求会带来延迟。

页面开头的 What-If 设问就是为了让你体会这个权衡: 如果系统是单体的, 换方言几乎不可能; 如果是流水线, 只需替换 TTS 环节即可。

二、架构设计

解耦与编排：三阶段流水线

[解释]

章节定位：流水线蓝图

本章展示 VLM -> LLM -> TTS 三阶段流水线的整体架构图和数据流向，是后续代码实战的设计依据。

多模型流水线：架构蓝图

智能体核心：流水线编排 (Pipeline Orchestration)

在本次架构升级中，我们复用 L25 的视觉感知模块，并依次接入文本推理与语音合成模块，形成数据流转闭环：

1. **第一阶段 (感知层)**：Qwen3-VL（已完成）。读取黑客松图片，提取算式，得出“裸答案”（例如：“12”）。
2. **第二阶段 (推理层)**：DeepSeek-V4-Flash。接收第一阶段的原始计算结果，结合特定 Prompt，生成拟人化的播报稿（例如：“同学你好，这道题的计算结果是12哦！”）。
3. **第三阶段 (行动层)**：CosyVoice2。接收第二阶段的口语文本，生成自然流畅的流式音频，并进行播报。

数据流向：

Hackathon Image -> [VLM API] -> "12" -> [LLM API] -> "答案是12哦" -> [TTS API] -> Audio Stream

[解释]

三阶段流水线的数据流

本页是整节课的核心架构图。数据从左到右流动：黑客松图片 → VLM 输出裸答案"12" → DeepSeek 输出播报文案"答案是12哦" → CosyVoice 输出音频流。

注意三个关键设计决策：(1) 每个阶段的输出就是下一个阶段的输入，形成严格的数据传递链；(2) 阶段之间通过标准的文本字符串衔接，不存在私有格式；(3) 我们的 FastAPI 代码扮演的是“总调度”角色，它负责按顺序调用三个云端模型并传递中间结果。

三、代码实战

在原有系统上集成新模块

【解释】

章节定位：动手编码

本章在 L25 的代码基础上做增量开发，分两步接入 DeepSeek 文本推理和 CosyVoice 语音合成模块。

步骤一: 集成文本推理模块 (DeepSeek)

在 L25 的 `analyze_url` 路由内部, 获得 VLM 的答案后, 我们不直接返回, 而是立即将其交给 `DeepSeek` 进行加工:

```
# (接L25代码...)
vlm_answer = resp.choices[0].message.content.strip()

# 接入第二棒: 语言润色
llm_resp = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V4-Flash",
    messages=[
        {"role": "system", "content": "你是一个充满活力的AI学伴。请根据传入的答案, 生成一句简短的语音播报稿。不要多余的废话。"},
        {"role": "user", "content": f"VLM解题结果是: {vlm_answer}"}
    ]
)
broadcast_script = llm_resp.choices[0].message.content
```

生态一致性: SDK 的平滑挂载

得益于 SiliconFlow 提供的统一 OpenAI SDK, 调用不同的模型 (Qwen 与 DeepSeek) 的语法完全一致。在原有代码中增加这段逻辑, 成本很低。

[解释]

DeepSeek 集成的代码要点

本页展示了在 L25 代码上追加的第一段新逻辑。核心改动只有一处: 在拿到 VLM 的 `vlm_answer` 后, 不直接返回, 而是再发一次 `client.chat.completions.create()` 请求。

注意两个细节: (1) `model` 参数换成了 `deepseek-ai/DeepSeek-V4-Flash`, 但 `client` 对象不用换, 因为 DeepSeek 和 Qwen 都通过同一个 SiliconFlow 平台提供服务, 共享同一个 `base_url`; (2) 这次请求使用了 `system` 角色消息来设定 DeepSeek 的行为——“你是一个 AI 学伴, 请生成播报稿”。这就是 Prompt 工程在流水线中的应用。

步骤二: 集成语音合成模块 (CosyVoice2)

拿到播报文案后，我们在后端新增一个专门的 TTS 流式接口 `/api/speak`，前端在收到文本后，自动请求该接口进行音频播放：

```
import requests
from fastapi.responses import StreamingResponse

# 第三棒：语音合成（行动/表达层）
@app.post("/api/speak")
async def speak(data: dict):
    # 直接请求原生 TTS API
    response = requests.post(
        "https://api.siliconflow.cn/v1/audio/speech",
        headers={"Authorization": f"Bearer {SILICONFLOW_API_KEY}"},
        json={
            "model": "FunAudioLLM/CosyVoice2-0.5B",
            "input": data["script"],
            "voice": "alex"
        },
        stream=True # 开启流式响应降低延迟
    )
    return StreamingResponse(
        response.iter_content(1024), media_type="audio/mpeg"
    )
```

🤔 架构反思 (What-If):

Q: 为什么前两步都用方便的 OpenAI SDK，这里却要退回使用原生的

`requests` 发送 HTTP 请求？

A: 目前业界绝大多数标准化大模型 SDK 的核心规范围绕“文本/JSON 生成”。而对于需要实时接收并处理二进制音频流

（`stream=True`）的特定场景，直接使用底层的 `requests` 库是兼容性最强、最透明的微服务通信方案。这也说明了一个工程原则：不盲从工具，而是根据数据流的实际特性选择最合适的方案。[逻辑补强]

[解释]

TTS 集成与接口拆分

本页有两个知识点。第一，为什么这里不用 OpenAI SDK 而退回 `requests`？因为 TTS 返回的是二进制音频流，不是 JSON 文本。OpenAI SDK 的标准接口主要围绕文本/JSON 生成设计，处理二进制流时直接用 `requests` 的 `stream=True` 更简单可靠。

第二，为什么要把 TTS 拆成独立的 `/api/speak` 接口？因为 VLM + DeepSeek 的推理可能需要数秒。如果把语音合成也塞进同一个请求里，用户要等所有环节全部完成才能看到任何反馈。拆分后，前端可以先显示文字结果，再异步请求语音接口实现“边下边播”，用户体验显著提升。

随堂架构反思 (Architecture Review)

Q: 在上述流水线中，如果前端网页的文字已经迅速显示，但音频播放却出现严重卡顿或无声，从架构视角看，应首先排查哪个环节？

- A. 检查 Qwen3-VL 的图片解析并发量是否超载
- B. 检查 DeepSeek 文本推理阶段的 API 响应延迟
- C. 检查 FastAPI /api/speak 接口中 stream=True 是否生效及网络层状态
- D. 重启整个 Python 后端服务并刷新前端

► 点击查看解析

[解释]

故障隔离与排查逻辑

这道选择题考查的是流水线架构中"故障边界隔离"的概念。题目描述的现象是"文字正常但语音异常"。

既然文字已经正常显示，说明 VLM（第一阶段）和 DeepSeek（第二阶段）都已成功返回。因此故障一定被隔离在第三阶段——TTS 接口或音频传输。选项 A 和 B 排查的是上游模块，与现象矛盾；选项 D 是"重启大法"，不是工程思维。

这就是模块化流水线架构的核心优势之一：当某个环节出问题，可以精确定位到具体模块，而不需要从头到尾排查整个系统。

四、自适应能力验证

外部环境变化与系统泛化

[解释]

章节定位：验证架构

本章通过更换更难输入题目来测试流水线的泛化能力，验证模块化架构在环境变化时是否依然稳定。

步骤三：验证流水线的泛化与鲁棒性

挑战：改变输入，不改代码

黑客松平台升级了难度。现在请大家在前端将题库获取接口加上一个高级参数：

```
https://syb.ncu.edu.cn/smart-class/api/hackathon/vision/puzzle?level=advanced
```

原来简单的 3×4 算术题，变成了需要解 x 的一元一次方程。

测试要求：

在不修改任何后续推理与合成模块代码的前提下，重新执行流水线。

系统泛化表现验证

测试结果表明：视觉感知模块

（VLM）成功解析了一元一次方程图片；文本推理模块（LLM）根据输入复杂度的变化，自适应地生成了更严谨或更具鼓励性的解释话术；语音合成模块（TTS）则稳定输出了对应的流式音频。

结论分析：

这体现了基于大模型的流水线架构的核心优势。系统在面对未编程的边缘情况（Edge Cases）时，依然能正确处理，无需修改代码。

[解释]

泛化能力验证的工程意义

本页的实验设计非常简单但意义重大：把输入从简单算术题换成一元一次方程，观察系统表现。关键约束是“不修改任何代码”。

实验结果表明：VLM 能识别新类型的图片，DeepSeek 会根据答案复杂度自动调整措辞，TTS 稳定输出音频。整条流水线无需人工干预即可适应输入变化。

这种能力来自大模型本身的泛化特性——它们不是按规则匹配输入，而是通过语义理解来处理未见过的内容。传统的规则式程序做不到这一点。这也说明，当核心架构设计合理时，系统的适应能力主要取决于模型质量，而非代码量。

五、小结

从单体脚本到模块化流水线

[解释]

章节定位：知识收敛

本章总结可插拔架构和泛化验证两大核心收获，并预告下一讲的 Chrome 插件开发。

总结与预告

模块化解耦的力量

今天，我们在 L25 的单体代码基础上，以极低的重构成本集成了文本推理与语音合成模块。

1. **可插拔架构**：任何一个节点的变更（如升级视觉大模型或更换方言 TTS），都不影响上下游代码的运作。
2. **系统鲁棒性**：面对外部输入复杂度的显著升级，流水线系统展现出了优秀的泛化能力。

工业级智能体架构

从依赖单一模型，走向**多模型协作流水线 (Multi-Model Pipeline)**，是工业界解决复杂业务的主流选择。

这要求我们的思维从“调通一段脚本”转变为“设计一个健壮的系统”。低耦合、高可扩展性——这就是真实的软件工程。

下一讲预告：

目前我们的智能体还停留在服务器的终端里。下一课，我们将开发一个驻留在浏览器中的 Chrome 插件智能体，让 AI 能力直接嵌入用户的日常工作流。

[解释]

本讲知识体系回顾

本讲完成了一次“增量式架构升级”：在 L25 的单体代码上，只追加了两段逻辑（DeepSeek 润色 + CosyVoice 语音），就把一个只会输出数字的程序升级为会说话的智能伙伴。

核心收获有两点：(1) 可插拔架构——任何一个节点都可以独立替换，不影响上下游。比如把 CosyVoice 换成另一个 TTS 引擎，只需改 /api/speak 的实现，前面的 VLM 和 DeepSeek 代码完全不用动。(2) 泛化验证——换了更难的输入，代码不改，系统依然能正常工作。

下一讲（L27）将把智能体从服务器终端搬进浏览器，开发 Chrome 插件。